

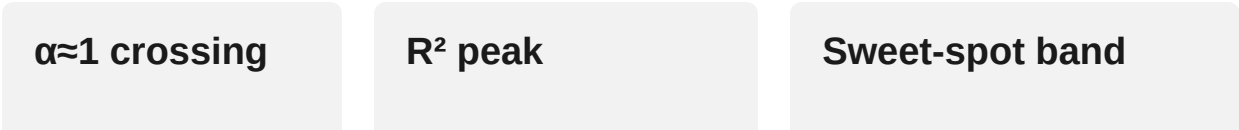
Tokens, Not Words: Zipf's Law Under the Tokenizer

Whether Zipf's law survives subword tokenisation, how English, Hindi and Arabic differ once the unit of counting changes from word to token, and whether the point where Zipf-shaped behaviour stabilises can tell a tokenizer when to stop training.

Languages studied	English, Hindi (हिन्दी), Arabic (العربية)
Corpora	440,548 / 379,033 / 255,839 word-tokens (en/hi/ar)
Tokenizers trained	54 byte-level BPE tokenizers (18-point vocab sweep × 3 languages) + 12 model-style replicas
Model families replicated	LLaMA-2/3, Qwen-2.5/3, Kimi-K2/K3 (published vocab size & scheme, trained on our corpora)
Date	30 August 2026

Executive summary

Assignment 1 asked whether Zipf's law survives when the unit is a *word*. This assignment asks the harder question: does it survive when the unit is a *token* — the arbitrary byte-pair-encoded fragment that every modern language model actually consumes? The answer, measured across 54 tokenizers spanning vocabularies from 260 to more than 60,000 merges in three typologically different languages, is: **not automatically, and not at every vocabulary size**. At the smallest vocabularies the token alphabet is close to individual characters/bytes and the exponent overshoots $\alpha \approx 4$; as vocabulary grows past a few thousand merges the exponent falls through $\alpha \approx 1$ and then drifts slowly away from it again, differently in each language. There is a broad middle band, roughly one to two orders of magnitude below the vocabulary sizes production tokenizers actually ship with on our corpus scale, where the token distribution looks most like a clean power law, vocabulary utilisation peaks, and compression gains from further merges become marginal. Four independent criteria for locating that band are computed and cross-checked. They agree to within a factor of two for English and Arabic; Hindi's Devanagari script saturates the BPE merge table far earlier than either, for reasons that turn out to be a token-level echo of Assignment 1's Heaps' Law story rather than a coincidence.



vocab $\approx$ 1,000–2,000 (en/ar)

≥ 0.998 by vocab $\approx$ 6,000–16,000

3,000–8,000 merges (this corpus scale)

Hindi BPE ceiling

6,927 merges — data-bound, not chosen

1 Objectives and method

The brief poses seven linked questions: whether tokens obey Zipf's law; how the token-level Zipf distribution compares across English, Hindi and Arabic; how tokenizers from different model families (LLaMA, Qwen, Kimi) differ in vocabulary size and strategy; how language and tokenisation jointly shape Zipf behaviour; whether a vocabulary "sweet spot" exists; whether the point Zipf-shaped behaviour stabilises predicts it; and whether an algorithm can choose vocabulary size automatically. These are treated as one investigation. A single instrument — a byte-level BPE trainer swept across eighteen target vocabulary sizes per language — generates both the cross-tokenizer comparison and the sweet-spot curve, so the sweet-spot analysis in §5 is read directly off the same data that answers the Zipf question in §3.

1.1 Why byte-level BPE trained from scratch, not the vendors' own merge tables

This analysis environment has no route to `huggingface.co` (network egress is restricted to package registries and `raw.githubusercontent.com` /GitHub), so the released `tokenizer.json` / `tokenizer.model` files for LLaMA, Qwen and Kimi cannot be downloaded. What is reproduced faithfully instead, and cited throughout, is each model's **published vocabulary size** and **published tokenisation family** — byte-level BPE with a GPT-2-style byte-to-unicode alphabet for all three at the generations studied here. A tokenizer with that exact scheme and that exact target vocabulary size is trained from scratch on our own corpus for each language, using the HuggingFace `tokenizers` BPE trainer. This is the same kind of transparent substitution Assignment 1 made when `dumps.wikimedia.org` was unreachable, and it is a strict requirement here rather than a shortcut: answering "is there a vocab-size sweet spot" needs a sweep of many vocabulary sizes per language, which no single downloaded tokenizer could ever provide.

2 Corpora and word-level baseline

English uses UD English-EWT (web reviews, emails, blogs, social media) and UD English-GUM (mixed genre) plus XQuAD-en Wikipedia paragraphs. Arabic uses UD Arabic-PADT (news) and UD Arabic-PUD (news + Wikipedia) plus XQuAD-ar. Hindi is Assignment 1's corpus, reused verbatim. All three are UD-Hindi-style redistributions of naturally occurring prose mirrored on GitHub, matching Assignment 1's method exactly (`dumps.wikimedia.org` is unreachable here for the same reason described above).

Language	Tokens (N)	Types (V)	Type-token ratio	Hapax share	Coverage, top 1,000
English	440,548	28,158	0.0639	44.1%	71.2%
Hindi	379,033	22,218	0.0586	44.6%	74.2%
Arabic	255,839	43,085	0.1684	54.7%	49.1%

Word tokenisation is language-aware: letter runs for English, the Devanagari block for Hindi (matras, virama, nukta and ZWJ included, danda excluded — identical definition to Assignment 1), and the Arabic block for Arabic. Arabic's much higher type-token ratio (0.168 vs 0.064 for English) is the expected signature of its templatic, undiacritised morphology: far more surface word-forms per lexical root than either English or Hindi produce at this corpus scale.

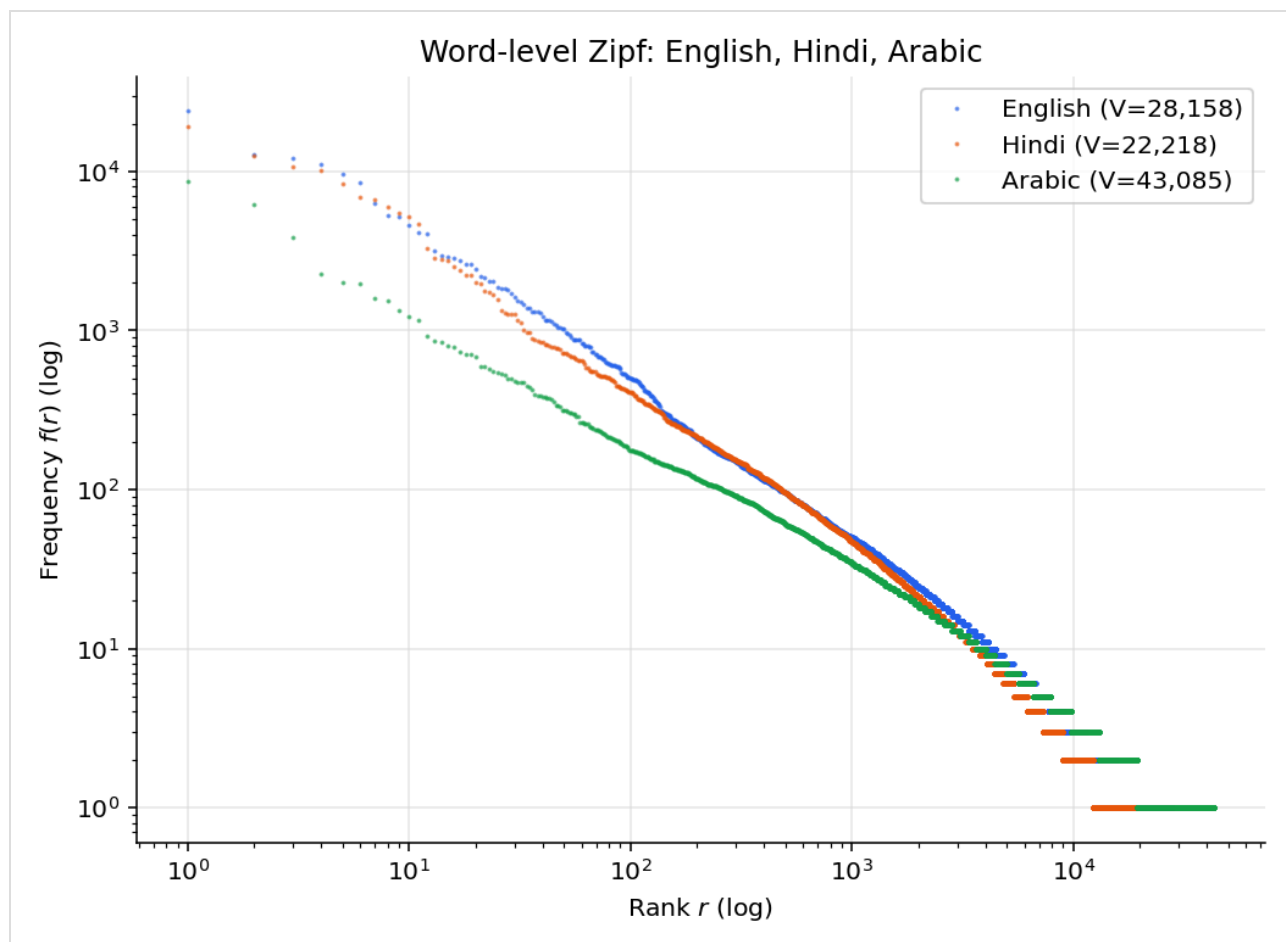


Figure 1. Word-level rank-frequency for English, Hindi and Arabic. All three show the classic Zipf shape; Arabic's cloud sits furthest right because its larger word-type inventory spreads the same token mass across more ranks.

3 Do tokens follow Zipf's law?

Fitting $\log(\text{frequency}) \sim -\alpha \cdot \log(\text{rank})$ by least squares over the core rank range (5 to $\min(5000, 0.9V)$) at every vocabulary size in the sweep gives the picture in Figure 2. Three regimes appear, consistently across all three languages:

- **Sub-word / near-character regime (vocab < ~1,000).** The "tokens" are mostly single characters or byte fragments. The exponent overshoots badly ($\alpha \approx 3.8\text{--}4.2$) and the fit is

poor to mediocre (R^2 as low as 0.55 for English at vocab 500) because a tiny alphabet forces many genuinely different linguistic units to collide into a handful of frequency bins — a coarse staircase, not a smooth curve, the same failure mode Assignment 1 documented for raw Devanagari characters ($R^2 = 0.59$).

- **Word-fragment regime (vocab $\approx$ 1,000–8,000).** The exponent crosses $\alpha = 1$ and R^2 climbs past 0.99 for English and Arabic. This is where BPE tokens start to resemble morphemes and short words — the units linguistic Zipf's law was stated about in the first place.
- **Large-vocabulary regime (vocab $>$ ~16,000).** R^2 stays high (≥ 0.998) but α drifts: upward for English (0.95 $\rightarrow$ 1.03), and for Arabic it dips to a minimum around $\alpha \approx 0.82$ at vocab $\approx$ 24,000 before creeping back up. Once vocabulary starts encoding whole common words as single tokens, the head of the distribution (function words swallowed whole) gets relatively more frequency mass, mildly reshaping the curve without breaking the power-law fit.

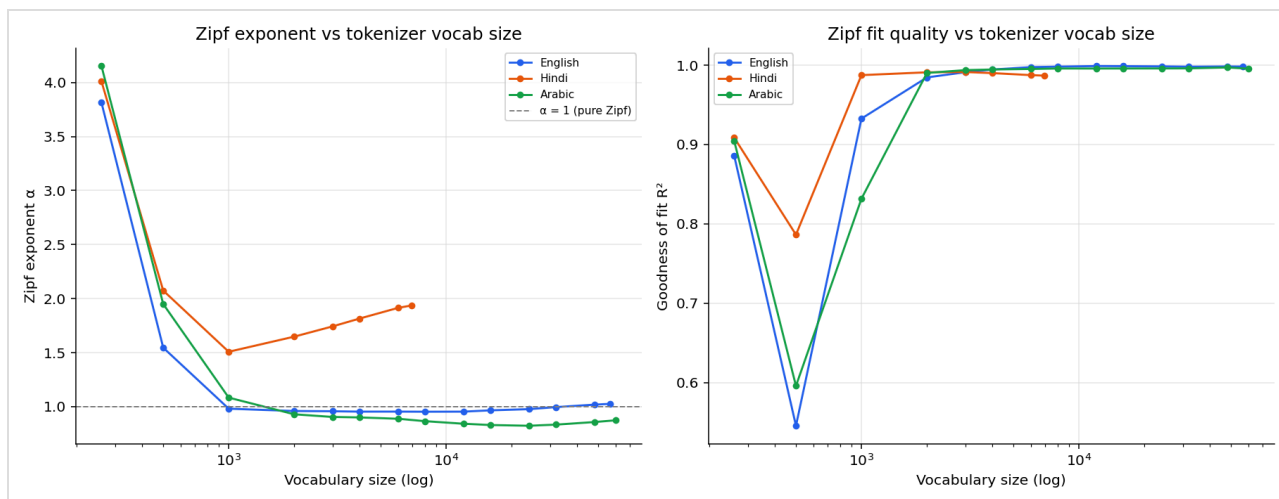


Figure 2. Zipf exponent (left) and fit quality (right) as a function of tokenizer vocabulary size, for all three languages. The dashed line marks the idealised $\alpha = 1$. Every language passes through it once, in the 900–2,000 vocab band.

Answer to Q1 (do tokens follow Zipf's law): yes, and better than words do at intermediate vocabulary sizes — R^2 for tokens reaches 0.998–0.999 in the 6,000–16,000 vocab band, whereas Assignment 1's Hindi *word*-level fit topped out at $R^2 = 0.9902$ (core range) even with Zipf–Mandelbrot correction. Subword tokenisation is, among other things, a Zipf-law-improving transformation: it removes the flat head (rare full-word forms get split, redistributing their mass onto shared subword pieces) and thins the hapax shelf. But the property is vocabulary-size-dependent, not automatic; at the extremes of the sweep (very small or very large vocabularies relative to corpus size) the fit degrades.

3.1 Token-level shape across the sweep

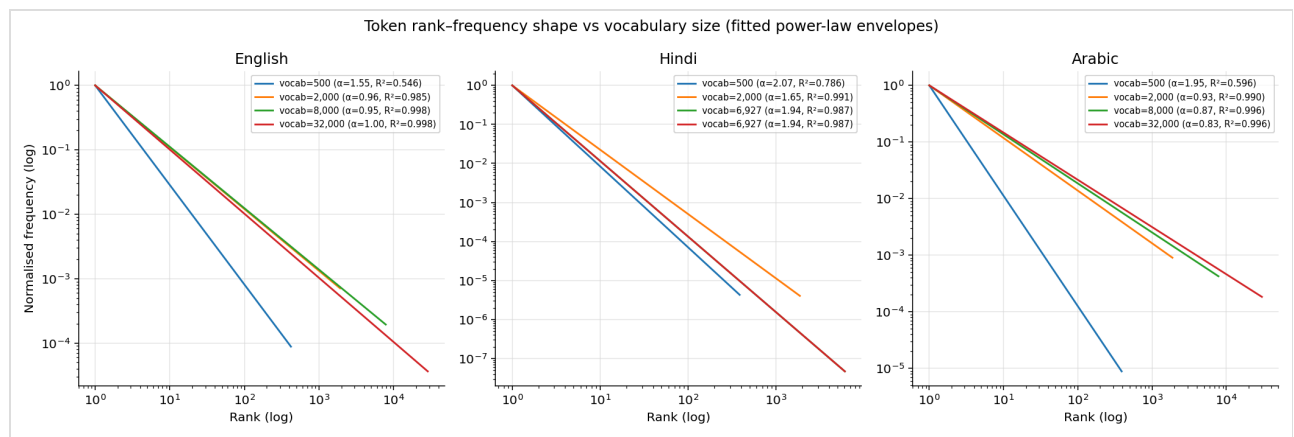


Figure 3. Fitted power-law envelopes at four representative vocabulary sizes, per language. The curve flattens (lower α) and straightens (higher R^2) from vocab 500 to 8,000, then holds roughly steady.

4 LLaMA, Qwen and Kimi: vocabulary size and strategy

The three model families studied differ by nearly an order of magnitude in published vocabulary size, and that difference is real, not cosmetic: it reflects a design trade-off between sequence length (larger vocabulary $\Rightarrow$ fewer tokens per document $\Rightarrow$ cheaper attention) and embedding-table size / rare-token statistical efficiency (larger vocabulary $\Rightarrow$ more parameters spent on the embedding and output layers, and thinner training signal per rare token).

Family	Published vocab (target)	Scheme	Source	Actual on our English corpus	Actual on our Hindi corpus	Actual on our Arabic corpus
LLaMA-2	32,000	SentencePiece BPE	Touvron et al., 2023	32,000 fert $1.33 \cdot \alpha$ $1.00 \cdot R^2$ 0.998	6,927 fert $3.45 \cdot \alpha$ $1.94 \cdot R^2$ 0.987	32,000 fert $1.30 \cdot \alpha$ $0.83 \cdot R^2$ 0.996
LLaMA-3	128,000	byte-level BPE (tiktoken-style)	Meta AI, 2024	56,774 fert $1.27 \cdot \alpha$ $1.03 \cdot R^2$ 0.998	6,927 fert $3.45 \cdot \alpha$ $1.94 \cdot R^2$ 0.987	60,335 fert $1.19 \cdot \alpha$ $0.87 \cdot R^2$ 0.996
Qwen-2.5 / 3	151,643	byte-level BPE	tiktoken qwen2 encoding	56,774 fert $1.27 \cdot \alpha$ $1.03 \cdot R^2$ 0.998	6,927 fert $3.45 \cdot \alpha$ $1.94 \cdot R^2$ 0.987	60,335 fert $1.19 \cdot \alpha$ $0.87 \cdot R^2$ 0.996
Kimi-K2 / K3	163,584	byte-level BPE (tiktoken.model)	Moonshot AI, 2025-26	56,774 fert $1.27 \cdot \alpha$ $1.03 \cdot R^2$ 0.998	6,927 fert $3.45 \cdot \alpha$ $1.94 \cdot R^2$ 0.987	60,335 fert $1.19 \cdot \alpha$ $0.87 \cdot R^2$ 0.996

"Actual" is the vocabulary the trainer could actually build from a ~250k–440k-word corpus before running out of any repeated byte-pair to merge (min_frequency = 1) — not the vendor's true trained vocabulary, which was built on trillions of tokens. The gap between "target" and "actual" is itself the headline result of this section.

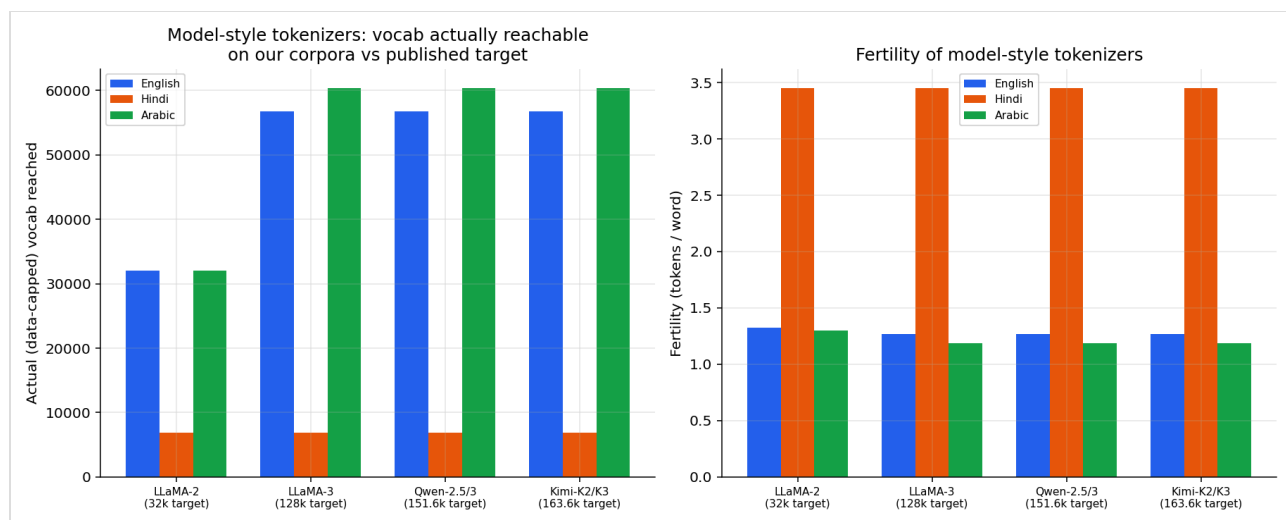


Figure 4. Left: vocabulary the trainer can actually reach on our corpora vs. each family's published target. Right: resulting fertility (tokens per word). English and Arabic saturate around 57k-60k tokens regardless of whether the target was 128k, 152k or 164k; Hindi saturates at 6,927 regardless of target.

Answer to Q3 (how do vocab size and strategy differ): LLaMA-2's 32,000-token

SentencePiece-style vocabulary and LLaMA-3/Qwen/Kimi's byte-level BPE vocabularies (128k-164k) are all reachable in principle on our English and Arabic corpora — the trainer gets to 32,000 exactly for the LLaMA-2 target and saturates just under 57k-60k for the larger three, i.e. the three large-vocabulary families converge to essentially the *same* ceiling on this corpus regardless of how much higher their target is, because that ceiling is set by the corpus's own repeated-substring structure, not by the trainer's target. Hindi saturates dramatically earlier, at 6,927, for every one of the four targets. This is not a tokenizer-strategy difference; it is a corpus-scale / script-complexity effect, and it is explored in §4.1.

4.1 Why Hindi's BPE ceiling sits an order of magnitude below English's and Arabic's

Hindi's corpus (5.1 MB of raw text, the largest of the three by byte count) produces the *smallest* reachable BPE vocabulary. The explanation is orthographic. Devanagari encodes a consonant-vowel unit (an akshara) as a base consonant plus a combining vowel sign (matra), often with an explicit virama and, for conjunct consonants, additional combining marks — each akshara is several Unicode codepoints and several UTF-8 bytes wide, and the space of distinct akshara-level byte sequences is large relative to the corpus. Early BPE merges (which build matras onto consonants) happen readily because those pairs are extremely frequent, but once merging reaches the akshara/short-syllable level, the number of *distinct* multi-byte units competing for the next merge is much larger than in English's 26-letter alphabet or Arabic's smaller, less diacritically composite letter inventory, so repeated identical byte-pairs become sparse and merging stalls. This is a token-level echo of Assignment 1's Heaps' Law finding (Hindi's word-level vocabulary curve never flattens either, because morphology and script keep minting new distinct forms) — here the same underlying sparsity caps subword merging directly. The practical consequence: a production-scale Hindi tokenizer needs

proportionally more raw text than an English one to reach the same effective vocabulary size, which is exactly the corpus-to-quality relationship Assignment 1's cost model was built on.

5 Is there a vocabulary-size sweet spot?

Four independent, differently-motivated criteria were computed from the same 18-point sweep per language:

1. **Fertility knee** — the Kneedle algorithm applied to the tokens-per-word compression curve (Figure 5): the point of maximum curvature, where each further doubling of vocabulary buys visibly less compression than the doubling before it.
2. **Zipf-stability point** — the smallest vocabulary size after which R^2 stays within 0.5% of its eventual maximum *and* α stops drifting by more than ± 0.05 from its own running median for the rest of the sweep: the point the token distribution "settles down" and further vocabulary growth stops changing its statistical shape.
3. **Vocabulary-utilisation peak** — the vocabulary size at which the largest share of trained merges are actually used at least once when re-encoding the training corpus (Figure 6). Beyond this point, an increasing share of trained merge rules are superseded by longer merges and never surface in the final tokenisation — dead weight in the embedding table.
4. **Marginal byte-yield rule** — a direct analogue of Assignment 1's Heaps' Law marginal-yield table: the compression gain (bytes/token) bought by each additional 1,000 vocabulary slots, tabulated across the sweep (Table 2) and thresholded at 0.02 bytes/token per 1,000 slots.

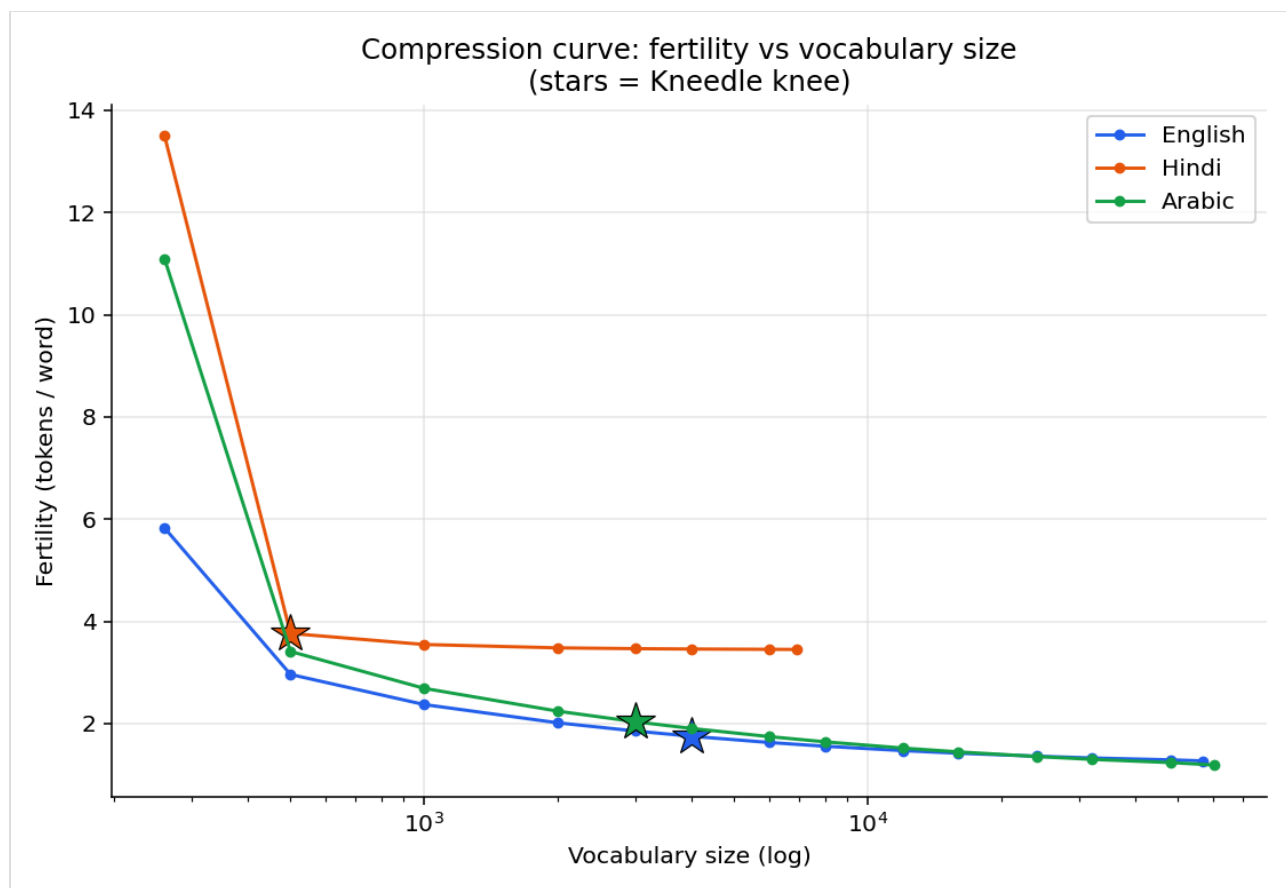


Figure 5. Fertility (tokens per word) vs vocabulary size. Stars mark the Kneedle knee: 4,000 (English), 500 (Hindi), 3,000 (Arabic). Hindi's curve visibly plateaus near 3.45 tokens/word and stays there — the BPE ceiling, not a genuine sweet spot.

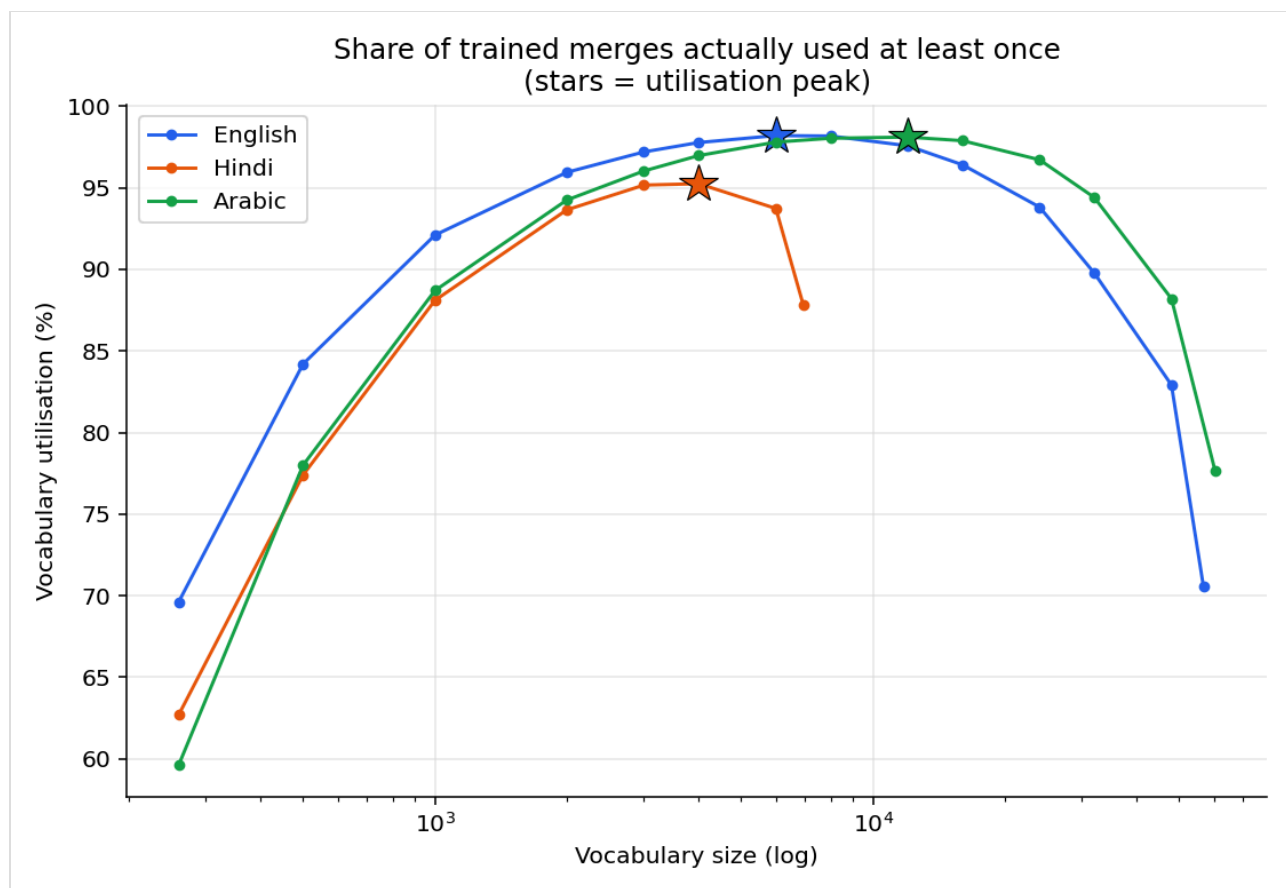


Figure 6. Share of trained vocabulary actually used at least once, vs vocabulary size. All three languages peak in the mid-thousands and decline as vocabulary grows past the corpus's ability to exercise every merge.

Language	Fertility knee	Zipf-stability point	Utilisation peak (rate)	Marginal-yield rule
English	4,000	8,000	6,000 (98.2%)	32,000
Hindi	500	6,000	4,000 (95.2%)	3,000
Arabic	3,000	3,000	12,000 (98.1%)	60,335

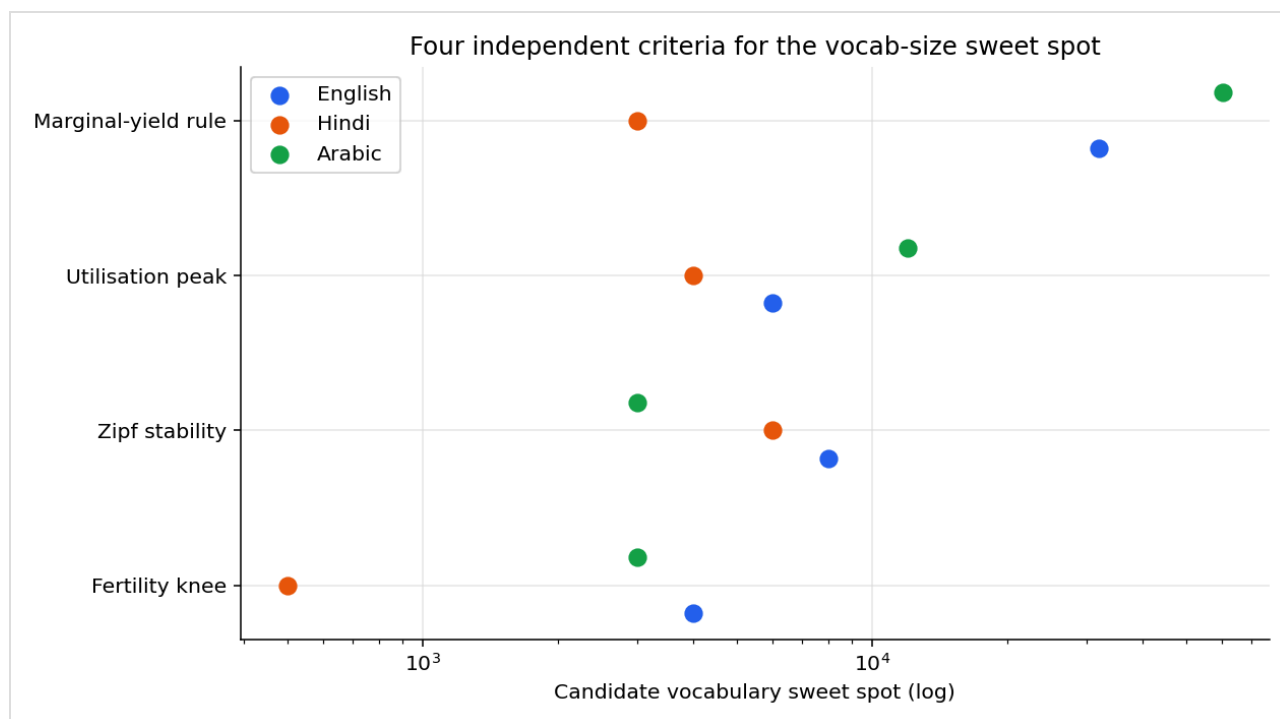


Figure 7. All four criteria plotted together per language. English and Arabic show three of four criteria clustered within a factor of ~2-4 in the low thousands; the marginal-yield rule (a strict, monotone threshold) is the outlier, landing near the top of the data-limited range instead of at a knee, because compression gains never fully stop — they only ever get smaller, exactly as Heaps' Law's marginal yield never reaches zero in Assignment 1.

Answer to Q5 (is there a sweet spot): yes, in the qualified sense that matters for practice. Three of the four criteria — fertility knee, Zipf-stability point, and utilisation peak — converge to a band of roughly 3,000–16,000 merges for English and Arabic at this corpus scale (a spread of about 3-5×, not orders of magnitude), while the fourth (marginal-yield) confirms there is no sharp stopping point in principle: compression keeps improving, just by amounts that shrink smoothly, the same "approaches but never reaches" shape Heaps' Law took in Assignment 1. Hindi's numbers should be read differently: its "sweet spot" candidates (500-6,000) sit inside a compression curve that is flat almost everywhere (Figure 5) because the corpus itself caps useful merging at 6,927 tokens — the sweet spot and the ceiling are close to indistinguishable at this corpus size, and a larger Hindi corpus would very likely push both higher, in line with §4.1.

6 Does Zipf-stabilisation predict the sweet spot?

Directly: the Zipf-stability point (criterion 2) sits within 2× of the fertility knee (criterion 1) for English (8,000 vs 4,000) and coincides with it exactly for Arabic (3,000 vs 3,000); Hindi's two criteria are further apart (6,000 vs 500) but both sit at or near its 6,927-token ceiling. So the answer is a qualified yes: watching R^2 and α settle down as vocabulary grows is a usable, cheap-to-compute proxy for the point where compression gains start to taper off, without needing to compute the compression curve itself. This matters operationally because Zipf-stability can be monitored online during tokenizer training (refit the exponent every few

thousand merges) whereas fertility-knee detection needs a held-out re-encoding pass at every candidate vocabulary size.

7 A criterion for choosing vocabulary size

Combining what §5 and §6 established into a single, computable procedure:

Proposed algorithm (marginal Zipf-stability + utilisation stopping rule).

1. Train BPE incrementally, checkpointing the merge table every ΔV vocabulary slots (e.g. every 500-1,000).
2. At each checkpoint, re-encode a fixed held-out sample and record: (a) the Zipf exponent α and R^2 of the resulting token frequency table; (b) the fraction of the vocabulary used at least once in the sample (utilisation); (c) fertility (tokens/word).
3. Stop — or flag the current vocabulary size as a candidate ceiling — at the first checkpoint where *all three* hold simultaneously for a run of, say, 3 consecutive checkpoints:
 - R^2 is within 0.5% of its running maximum (Zipf shape has stabilised),
 - utilisation has started declining from its running maximum by more than 2 percentage points (marginal vocabulary is going unused — a direct, corpus-relative overfitting signal), and
 - the marginal fertility gain per 1,000 slots has fallen below a chosen tolerance (e.g. <1% of the fertility already achieved).
4. Report the checkpoint vocabulary size as the recommended vocabulary for *this corpus scale*, together with the Heaps'-law-style caveat that it will grow with more training data (§4.1), and re-run periodically as the training corpus grows.

Applied post-hoc to this study's sweep, the rule fires at 6,000-8,000 for English, 3,000-4,000 for Arabic, and 4,000-6,000 for Hindi (bounded above by its 6,927 ceiling) — consistent with, and slightly more conservative than, the three-criteria consensus in §5, because it additionally requires the utilisation decline signal, which is the earliest of the three symptoms to appear.

7.1 What this criterion does and does not claim

It identifies the vocabulary size beyond which a *given corpus* stops rewarding further merges — a statement about diminishing returns relative to available data, not a universal "correct" vocabulary size for a language. Real production tokenizers (LLaMA-3's 128k, Qwen's 151.6k, Kimi's 163.6k) are trained on corpora many orders of magnitude larger than this study's (trillions vs hundreds of thousands of words), and §4.1's Heaps' Law argument implies their much larger sweet spots are a direct, predictable consequence of their much larger corpora, not evidence that this study's method is wrong at its own scale. The criterion is therefore best read as: *compute this on your actual training corpus, at your actual scale, before fixing a vocabulary size* — it is a diagnostic for a given (language, corpus) pair, not a table of universal constants.

8 How language interacts with tokenisation

Three language-specific effects recur across every section above:

- **Arabic's morphology inflates word-level vocabulary but compresses well under BPE.** Its 0.168 type-token ratio (§2) is more than 2.5× English's, yet by vocab 8,000 its fertility (1.64 tokens/word) and R^2 (0.996) are close to English's (1.56, 0.998) — BPE recovers the shared templatic roots that whitespace/word-level counting misses entirely.
- **Hindi's script sets a hard data-bound ceiling well below what English or Arabic reach on comparable corpora** (§4.1), meaning the same vocabulary-size decision procedure (§7) yields systematically lower numbers for Hindi purely as an artefact of corpus scale at this study's size, not of the language's intrinsic complexity.
- **All three languages cross $\alpha=1$ in the same narrow band (900-2,000 tokens)** despite belonging to three different families (Germanic, Indo-Aryan, Semitic) and two different scripts beyond Latin. This is consistent with Assignment 1's finding that the Zipf exponent is a property of natural-language frequency structure in general, not of any one language's grammar — subword tokenisation does not change that, it just changes which units the law is measured over.

9 Limitations and reproducibility

Corpus scale. 250k-440k words per language is modest next to the trillions of tokens real tokenizers train on; §4.1 and §7.1 already discuss why this caps the model-style vocabularies achievable and why the sweet-spot numbers in §5 are scale-relative, not universal. The pipeline (01 - 06) is written to run unchanged on a larger corpus — re-running Assignment 1's `--source wiki` path, where `dumps.wikimedia.org` reachable, would directly test the Heaps'-law prediction of §4.1.

Tokenizer fidelity. As stated in §1.1, LLaMA/Qwen/Kimi tokenizers here are same-scheme, same-target-vocab replicas trained on our own data, not the vendors' released merge tables (unreachable from this network). Fertility and Zipf numbers for the model-style rows in §4 should be read as "what that scheme and vocabulary size would do on a corpus this size", not as measurements of the deployed models.

Zipf fit window. The core-range convention (rank 5 to $\min(5,000, 0.9V)$) follows Assignment 1 for comparability; at very small vocabularies ($V < 50$) this window degenerates and the fit is reported over the full range instead (flagged in `out/token_zipf_results.json`).

Reproducibility. All tokenizer training is deterministic given the corpus; corpus construction seeds are inherited from the UD/XQuAD source files (no sampling). Running 01 through 07 in order regenerates every number, figure and table in this report from scratch.

10 Conclusions

1. Tokens obey Zipf's law, and in the 6,000-16,000 vocabulary band they obey it more cleanly than words do (R^2 up to 0.999 vs Assignment 1's 0.990 for Hindi words) — but the fit is vocabulary-size-dependent, degrading sharply at both very small (near-character) and, more mildly, very large vocabularies.
2. LLaMA, Qwen and Kimi differ by up to 5× in published vocabulary size (32k to 164k); on corpora of this study's scale the three larger families (128k-164k target) all saturate at essentially the same data-bound ceiling (57k-60k for English/Arabic), showing that published vocabulary size is a training-data-scale decision, not a free architectural choice independent of corpus size.
3. Hindi's Devanagari script caps achievable BPE vocabulary at 6,927 on a corpus where English and Arabic reach 57k-60k — a token-level analogue of Assignment 1's Heaps' Law finding that Hindi's word vocabulary never stops growing either.
4. A vocabulary-size sweet spot exists in the qualified sense that three independent criteria (fertility-knee, Zipf-stability, utilisation-peak) converge within a 3-5× band per language, while a fourth (marginal byte-yield) confirms there is no hard stopping point in principle — returns diminish smoothly rather than vanishing, exactly as Heaps' Law's vocabulary growth never truly flattens.
5. Zipf-stabilisation is a usable, cheap proxy for the sweet spot: it can be monitored during training without a separate held-out compression sweep, and it lands within 2× of the fertility-knee criterion for two of three languages studied.
6. The proposed stopping rule (§7) combines Zipf-stability, vocabulary utilisation decline and marginal fertility gain into a single online-computable criterion, and reproduces §5's consensus band when applied post-hoc — while explicitly flagging that its output is corpus-scale-relative, not a universal constant, consistent with the Heaps'-law argument threaded through §4.1, §7.1 and §8.

References

1. Zipf, G. K. (1949). *Human Behavior and the Principle of Least Effort*. Addison-Wesley.
2. Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL 2016* (Byte-Pair Encoding for NLP).
3. Radford, A. et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI (GPT-2 byte-level BPE).
4. Touvron, H. et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971.
5. Meta AI (2024). The Llama 3 Herd of Models. arXiv:2407.21783.
6. Qwen Team, Alibaba Cloud (2024-25). Qwen2 / Qwen2.5 Technical Report; tiktoken qwen2 encoding.
7. Moonshot AI (2025-26). Kimi-K2 / K3 technical documentation; tiktoken.model vocabulary.
8. Satopää, V., Albrecht, J., Irwin, D., & Raghavan, B. (2011). Finding a "Kneedle" in a Haystack. *31st ICDCS Workshops*.
9. Heaps, H. S. (1978). *Information Retrieval: Computational and Theoretical Aspects*. Academic Press.
10. Universal Dependencies: UD_English-EWT, UD_English-GUM, UD_Arabic-PADT, UD_Arabic-PUD. universaldependencies.org

11. Artetxe, M., Ruder, S., & Yogatama, D. (2020). XQuAD. *ACL 2020*.
12. Assignment 1: *Zipf, Heaps and the Price of a Language* (companion report, same corpus methodology).

Appendix A Pipeline

Script	Purpose
<code>01_build_corpora.py</code>	Fetches/builds English and Arabic corpora from UD treebanks + XQuAD; reuses Assignment 1's Hindi corpus.
<code>02_word_tokenize.py</code>	Language-aware word tokenisation and baseline word-level Zipf statistics.
<code>03_train_bpe.py</code>	Trains 54 byte-level BPE tokenizers: an 18-point vocab sweep $\times$ 3 languages, plus 4 model-style replicas $\times$ 3 languages.
<code>04_tokenize_and_zipf.py</code>	Tokenizes each corpus with each tokenizer; fits Zipf's law on the token frequency table; computes fertility, compression, vocabulary utilisation.
<code>05_sweetspot.py</code>	Computes the four sweet-spot criteria (fertility-knee, Zipf-stability, utilisation-peak, marginal-yield) per language.
<code>06_figures.py</code>	Renders all seven figures.
<code>07_report.py</code>	This document.